

OpenAI - Symphony

AI 워크플로우 체험기

Project board를 코딩 에이전트의 제어 평면으로 바꾸는 구현 이야기

OpenAI Symphony 기반

GitHub Projects + TypeScript

OpenAI Symphony 기반 · GitHub Projects + TypeScript 구현체 소개



GitHub Stars

18k+

OpenAI / symphony

OpenAI Symphony란?

Project Management Board를 코딩 에이전트의 제어 평면으로 전환하는 오픈소스 스펙

"Project Management Board를 코딩 에이전트의 control plane으로 전환한다"

핵심은 제품이 아니라 SPEC.md다. 트래커, 워크스페이스, 에이전트 런타임을 하나의 반복 가능한 운영 흐름으로 묶는다.

SPEC.md

기술적 실체

Linear

Issue Tracker

PR +500%

3주 성과

18.3k

GitHub Stars

메모

- 독립 제품이라기보다, 구현 가능한 레퍼런스 스펙과 Elixir 구현체로 공개
- 현재 레퍼런스는 Linear를 사용하지만, SPEC 자체는 언어와 플랫폼에 독립적

왜 공개했나?

Codex 기반 오케스트레이션 패턴을 외부 구현 가능한 형태로 보여주기 위해

01

레퍼런스 구현 공유

Codex 기반 에이전트 오케스트레이션을 실제 코드와 SPEC으로 보여준다.

02

생태계 확장

다른 모델과 CLI가 같은 워크플로우 규칙 안에서 실행될 수 있다.

03

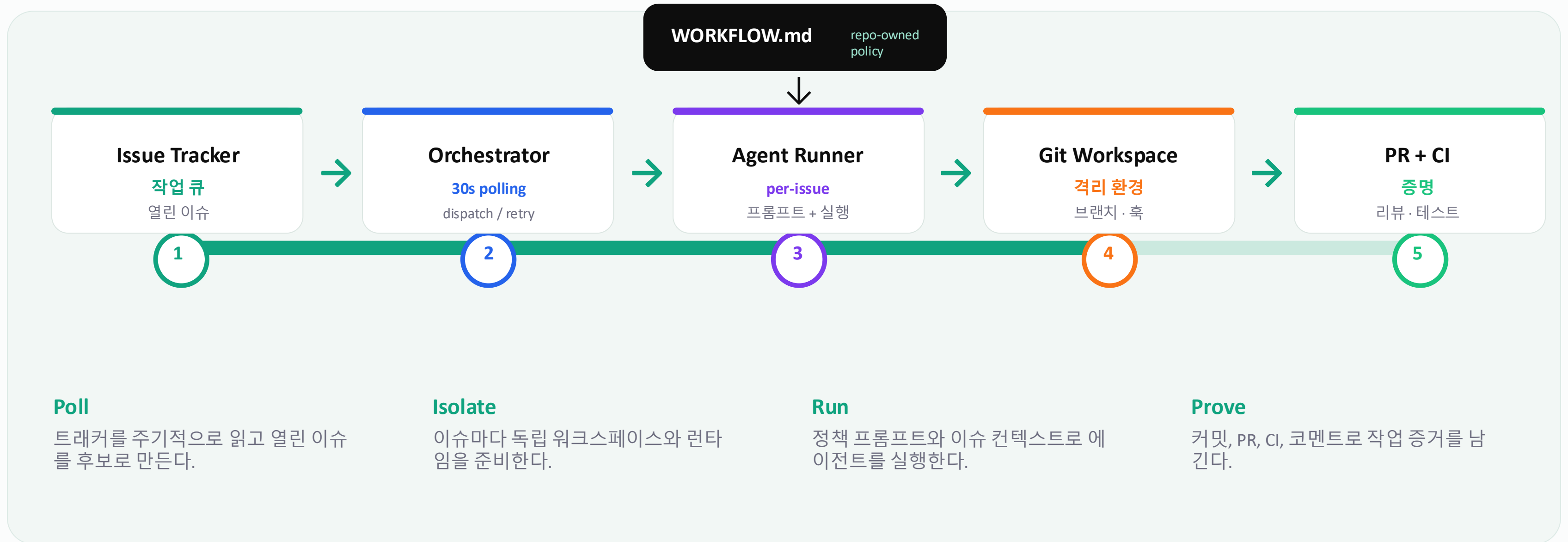
실무 검증 패턴 공개

내부 일부 팀에서 3주 만에 landed PR이 500% 증가한 운영 패턴이다.

핵심 메시지: Symphony는 “코딩 에이전트를 잘 쓰는 법”이 아니라 “일을 자동으로 흘러보내는 법”을 스펙으로 만든 사례다.

전체 구조

Issue Tracker에서 PR + CI까지 이어지는 실행 파이프라인



Orchestrator는 단일 권한자다. Agent는 실행자이고, 상태 전이는 트래커와 PR/CI가 증명한다.

핵심 컴포넌트 8가지

SPEC.md Section 3.1의 구현 단위

01

Workflow Loader

WORKFLOW.md 파싱

02

Config Layer

typed getter · env · validation

03

Issue Tracker Client

이슈 fetch · 도메인 모델 정규화

04

Orchestrator

poll tick · dispatch · retry

05

Workspace Manager

per-issue 디렉토리 · 훅

06

Agent Runner

프롬프트 빌드 · 실행 · 스트림

07

Status Surface

운영자용 상태 뷰

08

Logging

issue/session 컨텍스트 로그

Loader, Config, Tracker, Orchestrator, Workspace, Runner 등 각 컴포넌트의 경계 설정이 우선

추상화 레이어

역할 경계를 먼저 나누기

1	Policy Layer	WORKFLOW.md	팀별 티켓 처리 · 검증 · 핸드오프 규칙	사용자 정의 영역
2	Configuration Layer	Config Layer	front matter → typed runtime settings	
3	Coordination Layer	Orchestrator	폴링 루프 · 동시성 · 재시도 · 리콘실리에이션	Symphony 처리 영역
4	Execution Layer	Workspace + Runner	파일시스템 생명주기 · 에이전트 프로토콜	
5	Integration Layer	Issue Tracker Client	API 호출 및 트래커 데이터 정규화	
6	Observability Layer	Logging + Status	운영자 가시성	

"Symphony is easiest to port when kept in these layers"

내가 Github Symphony를 개발한 이유

같은 SPEC.md, 다른 구현체 · 다른 플랫폼

GitHub Projects가 좋다

- 코드와 PR이 이미 GitHub에 있다
- 추가 도구 없이 Projects를 작업 큐로 사용
- GraphQL API가 오케스트레이터 구현에 적합

GH

Elixir 모른다 → TypeScript로 하자

- 레퍼런스 구현은 Elixir / GenServer 패턴
- TypeScript는 진입장벽이 낮고 생태계가 넓다
- Octokit과 GitHub REST/GraphQL 생태계 활용

TS

SPEC을 지키면 구현 언어와 트래커는 바뀔 수 있다.

시행착오

SPEC을 정독하지 않았다

"SPEC.md를 처음부터 정독했어야 했다"

실수 1

Agent가 오케스트레이터인 줄 알았다

- SPEC을 끝까지 읽지 않고 구현 시작
- Agent 자체가 orchestration 역할을 맡는 설계로 흐름이 꼬임

실수 2

Web Dashboard까지 붙이며 크게 시작

- 처음부터 UI, 모니터링, 운영 뷰까지 붙이려 함
- MVP보다 큰 설계로 시간을 쓰고 여러 번 다시 만듦

교훈: Orchestrator ≠ Agent. 결국 SPEC이 설계의 전부였다.

openai-symphony vs github-symphony

같은 SPEC, 다른 구현체

항목	openai-symphony	github-symphony
언어	Elixir (GenServer)	TypeScript (Node.js)
Issue Tracker	Linear	GitHub Projects
에이전트	Codex / Kata CLI	Claude Code 등 설정 가능

차별점은 “GitHub Projects를 control plane으로 쓴다”와 “TypeScript 생태계로 옮겼다”에 있다.

편한 점

사람은 설계하고, 에이전트는 병렬로 실행한다

01

인지 부하 감소

5개 이상의 세션을 동시에 돌릴 수 있어 멀티태스킹이 실질적으로 가능해진다.

02

설계에 집중

이슈 분해, blocking 관계, 상태 전환이 개발 흐름의 핵심 인터페이스가 된다.

03

외부 관측 가능

커밋, 푸시, 이슈 코멘트로 조직 누구나 진행 상황을 확인할 수 있다.

04

휴대폰으로 업무 처리

PM Agent로 이슈 발행까지 자동화하면 장소와 기기 제약이 줄어든다.

불편한 점

자동화가 강해질수록 설계와 검증의 책임도 커진다

01

PR 전까지 브레이크 불가

이슈와 설계 문서가 부실하면 PR이 올라오기 전까지 중단하기 어렵다.
→상태를 강제로 바꾸거나, blocker 플래그 추가

02

CI/CD 없으면 효과 반감

내 컴퓨터에서 즉시 확인하지 못하므로
자동 검증 파이프라인이 중요하다.

03

이슈 분해 단위 설계 필요

PM.md로 이슈 작성 규칙과 분해 규칙을 먼저 정해야 한다.

04

검증할 것이 쏟아진다

PR이 많아질수록 리뷰 부담이 커지고 테스트 자동화가 필수가 된다.

시사점

SPEC.md 하나가 배포 파이프라인이 되는 시대

대 딸깍의 시대

SPEC.md 하나로 소프트웨어를 배포하는 새로운 패러다임.
설계 문서가 곧 실행 파이프라인이 된다.

대 워터폴의 시대

에이전트는 명세서대로 만든다.
그래서 설계와 테스트에 더 집중해야 한다.

AI Workflow가 새로운 먹거리

Symphony는 여러 AI Workflow 중 하나다. Agent가 많을수록 컴퓨팅 파워가 모자르다.
컴퓨팅과 인프라를 다룰 줄 아는 개발자가 유리해진다.

개발자는 코드 작성자에서 workflow designer로 이동한다.

github-symphony

SPEC.md로 소프트웨어를 배포하는 새로운 시대

github.com/hojinzs/github-symphony

Q&A

SPEC.md

Issue Tracker
→ Orchestrator
→ Agent Runner
→ PR + CI

control plane for agents